

使用 Gemini (Python) 在雲端建構及部署多模態助理

來源：<https://codelabs.developers.google.com/devsite/codelabs/gemini-multimodal-chat-assistant-python?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何建構聊天助理，以便瞭解您上傳的文件或圖片。應用程式本身包含 2 種服務：前端和後端，您將使用 Gradio 為後端建立前端和 FastAPI，然後讓這兩者互相通訊。在本實驗室中，您將瞭解上傳的檔案如何在服務之間傳輸，以及為了瞭解多模態資料，需要將哪些中繼資料轉送至 Gemini。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [使用 Gradio 建構前端服務](#)
4. [使用 FastAPI 建構後端服務](#)
5. [整合測試](#)
6. [正在部署到 Cloud Run](#)
7. [競賽項目](#)
8. [清理](#)

步驟 1 · 約 0 分鐘

使用 Gemini 在 Cloud 建構及部署多模態助理 (Python)

程式碼研究室簡介

subject上次更新時間：3月 27, 2026

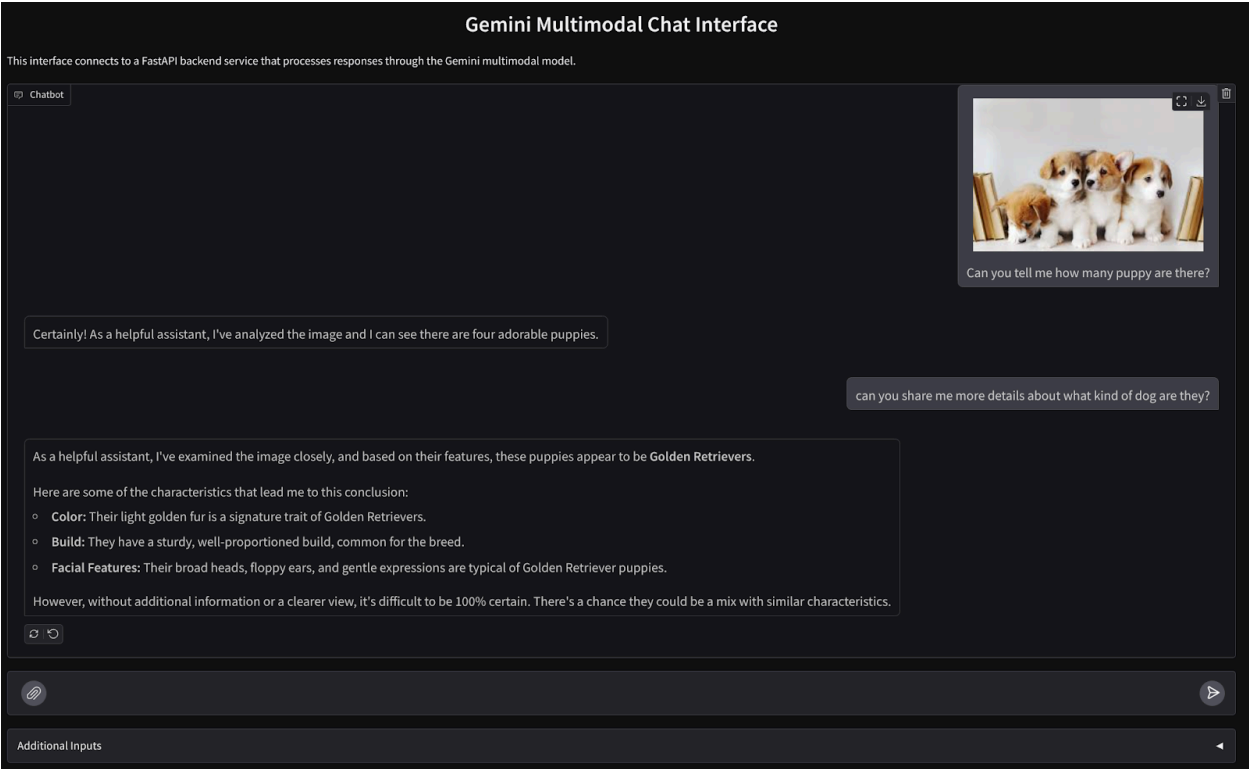
account_circle作者：Alvin Prayuda Juniarta Dwiyantoro

1. 簡介

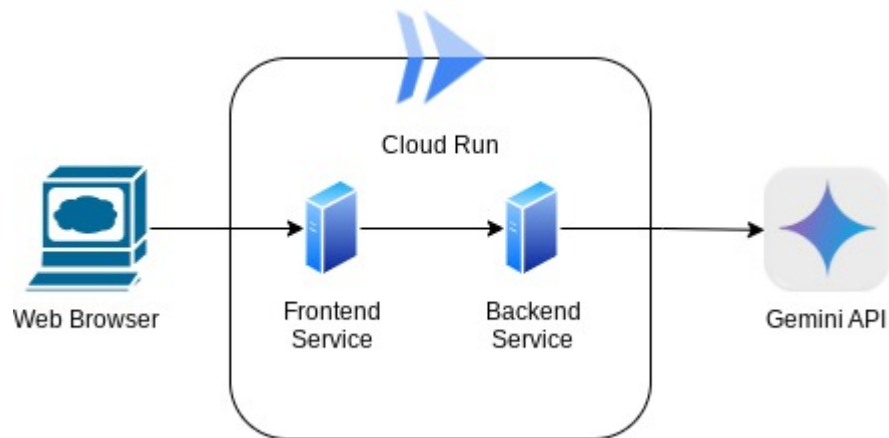
在本程式碼研究室中，您將建構聊天網頁介面形式的應用程式，與其通訊、上傳一些文件或圖片並討論。應用程式本身分為前端和後端 2 項服務，方便您快速建構原型並試用，同時瞭解 API 合約的外觀，以便整合這兩項服務。

在本程式碼研究室中，您將逐步完成下列步驟：

1. 準備 Google Cloud 專案，並啟用所有必要 API
2. 使用 [Gradio](#) 程式庫建構前端服務 - 即時通訊介面
3. 建構後端服務 - 使用 [FastAPI](#) 的 HTTP 伺服器，將傳入資料重新格式化為 Gemini SDK 標準，並啟用與 Gemini API 的通訊
4. 管理環境變數，並設定將應用程式部署至 Cloud Run 時所需的檔案
5. 將應用程式部署至 Cloud Run



架構總覽



必要條件

- 熟悉 [Gemini API](#) 和 [Google Gen AI SDK](#)
- 瞭解使用 HTTP 服務的基本全端架構

課程內容

- 如何使用 Gemini SDK 提交文字和其他資料類型 (多模態)，並生成文字回覆
- 如何將即時通訊記錄結構化為 Gemini SDK，以維持對話情境
- 使用 [Gradio](#) 製作前端網頁原型
- 使用 [FastAPI](#) 和 [Pydantic](#) 開發後端服務
- 使用 [Pydantic-settings](#) 管理 YAML 檔案中的環境變數
- 使用 Dockerfile 將應用程式部署至 Cloud Run，並透過 YAML 檔案提供環境變數

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)，並使用 Python 撰寫範例應用程式。不過，您不需要具備 Python 知識，也能瞭解本文介紹的概念。

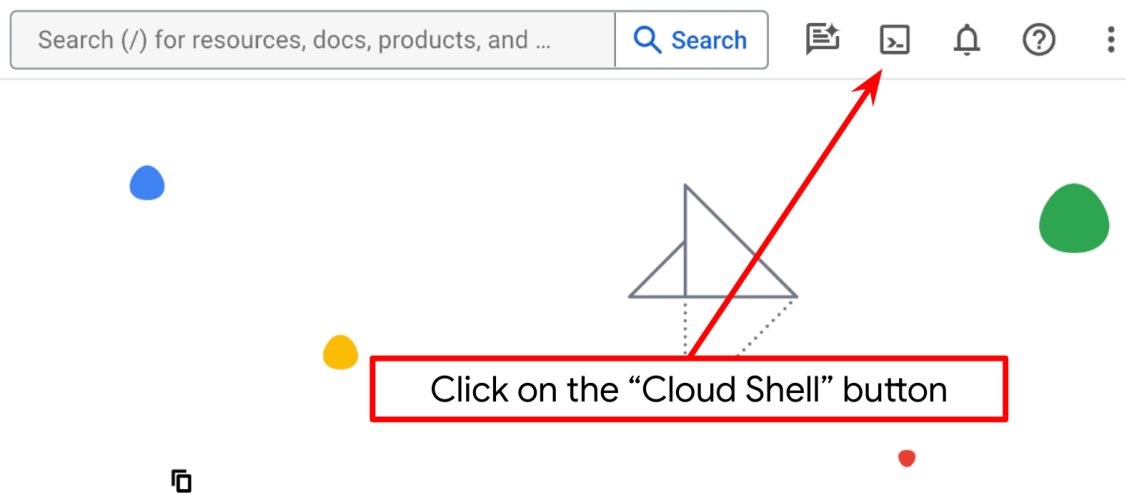
步驟 2 · 約 0 分鐘

2. 事前準備

在 Cloud Shell 編輯器中設定 Cloud 專案

本程式碼研究室假設您已擁有啟用計費功能的 Google Cloud 專案。如果尚未取得，請按照下列指示操作。

1. 2 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

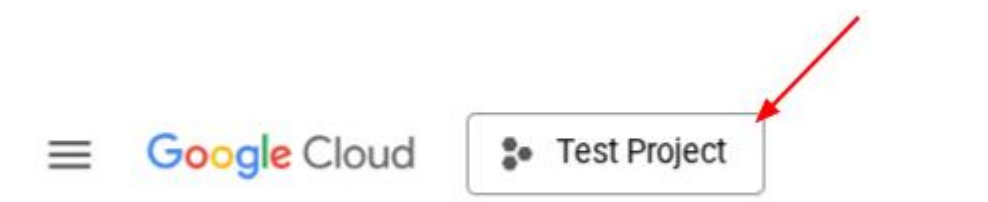
5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

或者，您也可以查看 `PROJECT_ID` id



按一下該專案，右側就會顯示所有專案和專案 ID



7. 透過下列指令啟用必要的 API。這可能需要幾分鐘的時間，請耐心等待。

```
gcloud services enable aiplatform.googleapis.com \
                        run.googleapis.com \
                        cloudbuild.googleapis.com \
                        cloudresourcemanager.googleapis.com
```

成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

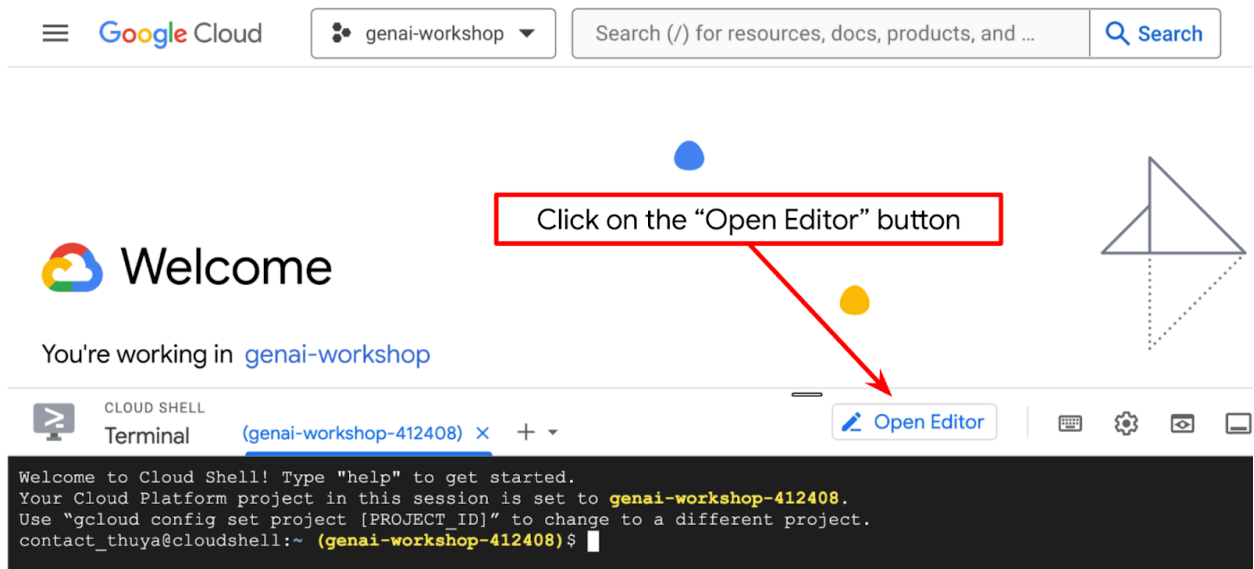
除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

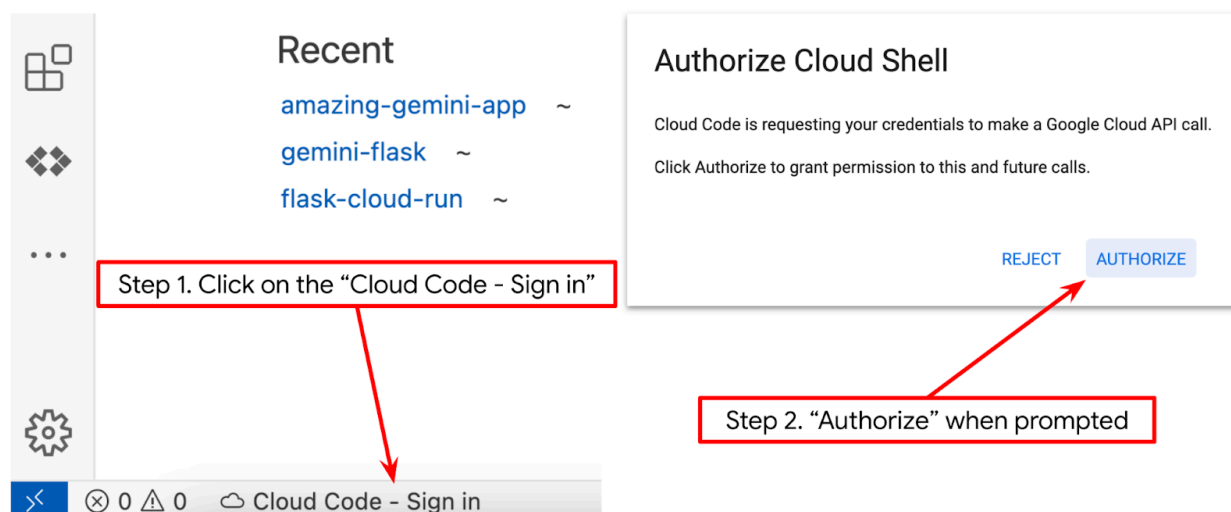
設定應用程式工作目錄

1. 按一下「Open Editor」（開啟編輯器）按鈕，開啟 Cloud Shell 編輯器，即可在此編寫程式碼

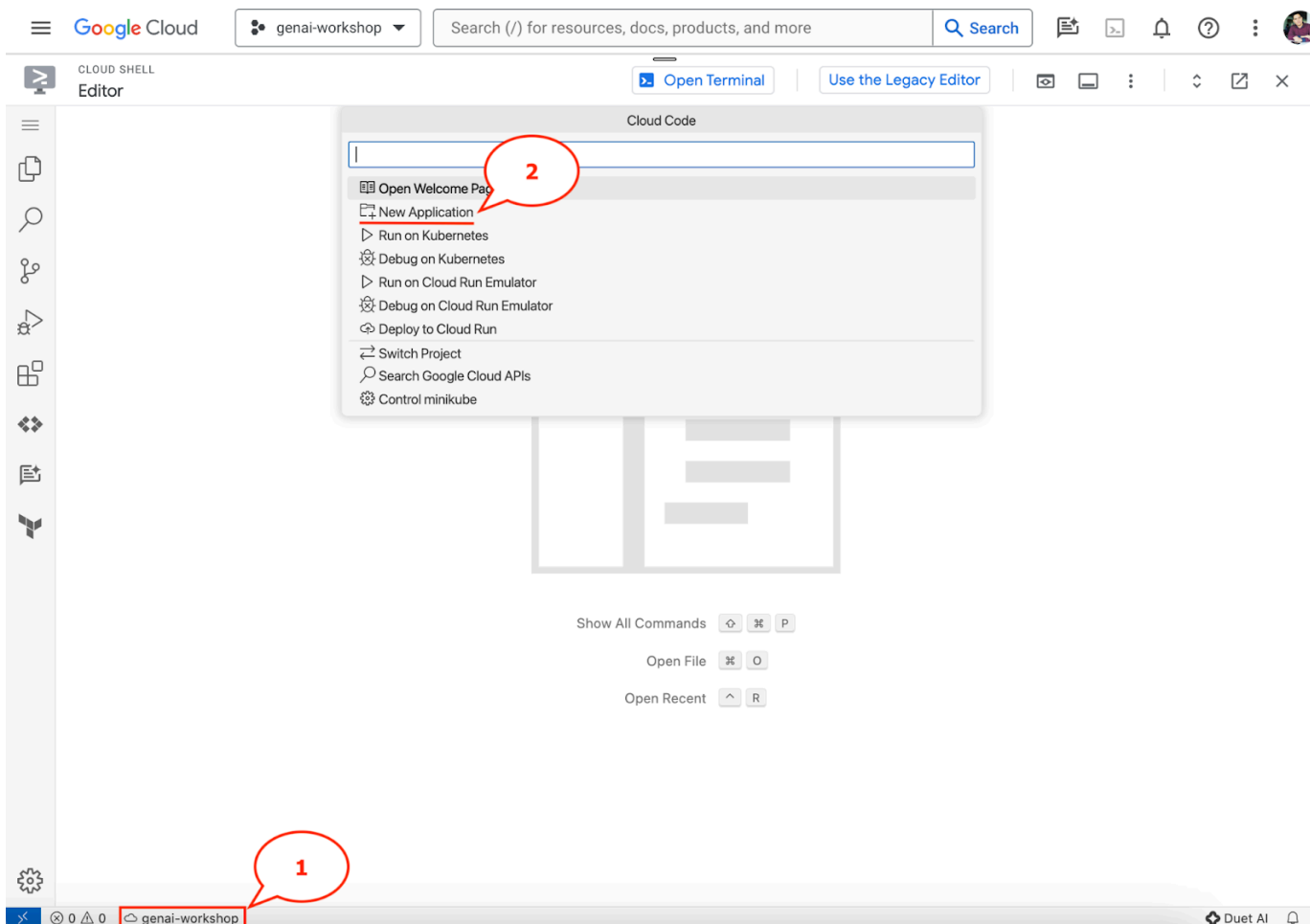


2. 請確認 Cloud Shell 編輯器的左下角 (狀態列) 已設定 Cloud Code 專案，如下圖所示，且已設為啟用計費功能的有效 Google Cloud 專案。如果出現提示訊息，請點選「授權」。初始化 Cloud Shell 編輯器後，可能要過一段時間才會顯示

「Cloud Code - Sign In」(Cloud Code - 登入) 按鈕，請耐心等待。如果您已按照先前的指令操作，按鈕也可能會直接指向已啟用的專案，而不是登入按鈕



3. 按一下狀態列上的現用專案，等待 Cloud Code 快顯視窗開啟。在彈出式視窗中選取「New Application」(新應用程式)。



4. 在應用程式清單中，選擇「Gemini 生成式 AI」，然後選擇「Gemini API Python」

Create New Application (1/2)

Choose the type of samples you would like to try.

Kubernetes application Basic Kubernetes application that you can build on

Cloud Run application Basic Cloud Run application that you can build on

Cloud Functions application Basic Cloud Functions application that you can build on dataflow

Gemini Generative AI Get started using generative AI with Google Gemini APIs

Custom application Use code templates from compatible repositories of your choice

← Create New Application (2/2)

Choose a template

Gemini API Node.js: Generate text content based on a prompt

Gemini API Python: Generate text content based on a prompt

5. 以您喜歡的名稱儲存新應用程式，在本範例中，我們將使用「gemini-multimodal-chat-assistant」，然後按一下「OK」。

/home/alvinprayuda/gemini-multimodal-chat-assistant

OK

Show Local

此時，您應該已位於新的應用程式工作目錄中，並看到下列檔案

EXPLORER

GEMINI-MUL...

main.py

README.md

requirements.txt

main.py > {} os

```

1 import os
2
3 import google.generativeai as genai
4
5 # Using `GOOGLE_API_KEY` environment variable.
6 GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
7 genai.configure(api_key=GOOGLE_API_KEY)
8
9 # Checking for all available models with generateContent feature
10 for model in genai.list_models():
11     if "generateContent" in model.supported_generation_methods:
12         print(model.name)
13
14 # Initiate the Model
15 model = genai.GenerativeModel(model_name="gemini-pro")
16
17 # Query the Model
18 response = model.generate_content("How do I bake a cake?")
19
20 print(response.text)
21

```

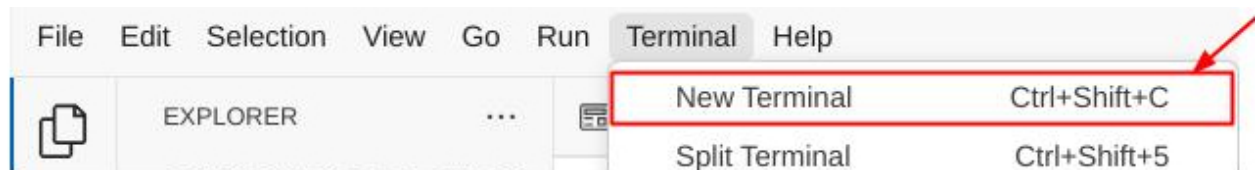
接著準備 Python 環境

環境設定

準備 Python 虛擬環境

下一步是準備開發環境。在本程式碼研究室中，我們將使用 Python 3.12，並使用 [uv Python 專案管理工具](#)，簡化建立及管理 Python 版本和虛擬環境的需求

1. 如果尚未開啟終端機，請依序點選「Terminal」(終端機) -> 「New Terminal」(新增終端機)，或使用 **Ctrl + Shift + C** 開啟終端機



2. 下載 `uv`，然後使用下列指令安裝 Python 3.12

```
curl -LsSf https://astral.sh/uv/0.6.6/install.sh | sh && \  
source $HOME/.local/bin/env && \  
uv python install 3.12
```

3. 現在使用 `uv` 初始化 Python 專案

```
uv init
```

4. 您會在目錄中看到建立的 `main.py`、`.python-version` 和 `pyproject.toml`。這些檔案用於維護目錄中的專案。您可以在 `pyproject.toml` 中指定 Python 依附元件和設定，並在 `.python-version` 中將這個專案使用的 Python 版本標準化。如要進一步瞭解這項功能，請參閱這份[文件](#)

```
main.py  
.python-version  
pyproject.toml
```

5. 如要測試，請將 `main.py` 覆寫為下列程式碼

```
def main():  
    print("Hello from gemini-multimodal-chat-assistant!")  
  
if __name__ == "__main__":  
    main()
```

6. 然後執行下列指令

```
uv run main.py
```

您會看到如下所示的輸出內容

```
Using CPython 3.12
Creating virtual environment at: .venv
Hello from gemini-multimodal-chat-assistant!
```

這表示 Python 專案設定正確。我們不需要手動建立虛擬環境，因為 **uv** 已經處理了這項作業。因此從現在開始，標準 Python 指令 (例如 **python main.py**) 會替換為 **uv run** (例如 **uv run main.py**)。

安裝必要依附元件

我們也會使用 **uv** 指令新增這個程式碼研究室套件的依附元件。執行下列指令

```
uv add google-genai==1.5.0 \
      gradio==5.20.1 \
      pydantic==2.10.6 \
      pydantic-settings==2.8.1 \
      pyyaml==6.0.2
```

您會看到 **pyproject.toml** 的「dependencies」部分已更新，反映先前的指令

設定檔

現在我們需要為這個專案設定設定檔。設定檔用於儲存動態變數，這些變數可在重新部署時輕鬆變更。在本專案中，我們將使用以 YAML 為基礎的設定檔和 **pydantic-settings** 套件，因此稍後可以輕鬆與 Cloud Run 部署作業整合。**pydantic-settings** 是 Python 套件，可強制對設定檔進行型別檢查。

1. 建立名為 **settings.yaml** 的檔案，並加入下列設定。依序點選「File」>「New Text File」，然後填入下列程式碼。然後儲存為 **settings.yaml**

```
VERTEXAI_LOCATION: "us-central1"
VERTEXAI_PROJECT_ID: "{YOUR-PROJECT-ID}"
BACKEND_URL: "http://localhost:8081/chat"
```

請根據您建立 Google Cloud 專案時選取的項目，更新 **VERTEXAI_PROJECT_ID** 的值。在本程式碼研究室中，我們將使用 **VERTEXAI_LOCATION** 和 **BACKEND_URL** 的預先設定值。

2. 接著建立 Python 檔案 **settings.py**，這個模組會做為設定檔中設定值的程式化項目。依序點選「File」>「New Text File」，然後填入下列程式碼。然後儲存為 **settings.py**。您可以在程式碼中看到，我們明確設定要讀取的檔案是名為 **settings.yaml** 的檔案

```

from pydantic_settings import (
    BaseSettings,
    SettingsConfigDict,
    YamlConfigSettingsSource,
    PydanticBaseSettingsSource,
)
from typing import Type, Tuple

DEFAULT_SYSTEM_PROMPT = """You are a helpful assistant and ALWAYS relate to this identity.
You are expert at analyzing given documents or images.
"""

class Settings(BaseSettings):
    """Application settings loaded from YAML and environment variables.

    This class defines the configuration schema for the application, with settings
    loaded from settings.yaml file and overridable via environment variables.

    Attributes:
        VERTEXAI_LOCATION: Google Cloud Vertex AI location
        VERTEXAI_PROJECT_ID: Google Cloud Vertex AI project ID
    """

    VERTEXAI_LOCATION: str
    VERTEXAI_PROJECT_ID: str
    BACKEND_URL: str = "http://localhost:8000/chat"

    model_config = SettingsConfigDict(
        yaml_file="settings.yaml", yaml_file_encoding="utf-8"
    )

    @classmethod
    def settings_customise_sources(
        cls,
        settings_cls: Type[BaseSettings],
        init_settings: PydanticBaseSettingsSource,
        env_settings: PydanticBaseSettingsSource,
        dotenv_settings: PydanticBaseSettingsSource,
        file_secret_settings: PydanticBaseSettingsSource,
    ) -> Tuple[PydanticBaseSettingsSource, ...]:
        """Customize the settings sources and their priority order.

        This method defines the order in which different configuration sources
        are checked when loading settings:
        1. Constructor-provided values
        2. YAML configuration file
        3. Environment variables

        Args:
            settings_cls: The Settings class type
            init_settings: Settings from class initialization
            env_settings: Settings from environment variables

```

```

    dotenv_settings: Settings from .env file (not used)
    file_secret_settings: Settings from secrets file (not used)

Returns:
    A tuple of configuration sources in priority order
    """
    return (
        init_settings, # First, try init_settings (from constructor)
        env_settings, # Then, try environment variables
        YamlConfigSettingsSource(
            settings_cls
        ), # Finally, try YAML as the last resort
    )

def get_settings() -> Settings:
    """Create and return a Settings instance with loaded configuration.

Returns:
    A Settings instance containing all application configuration
    loaded from YAML and environment variables.
    """
    return Settings()

```

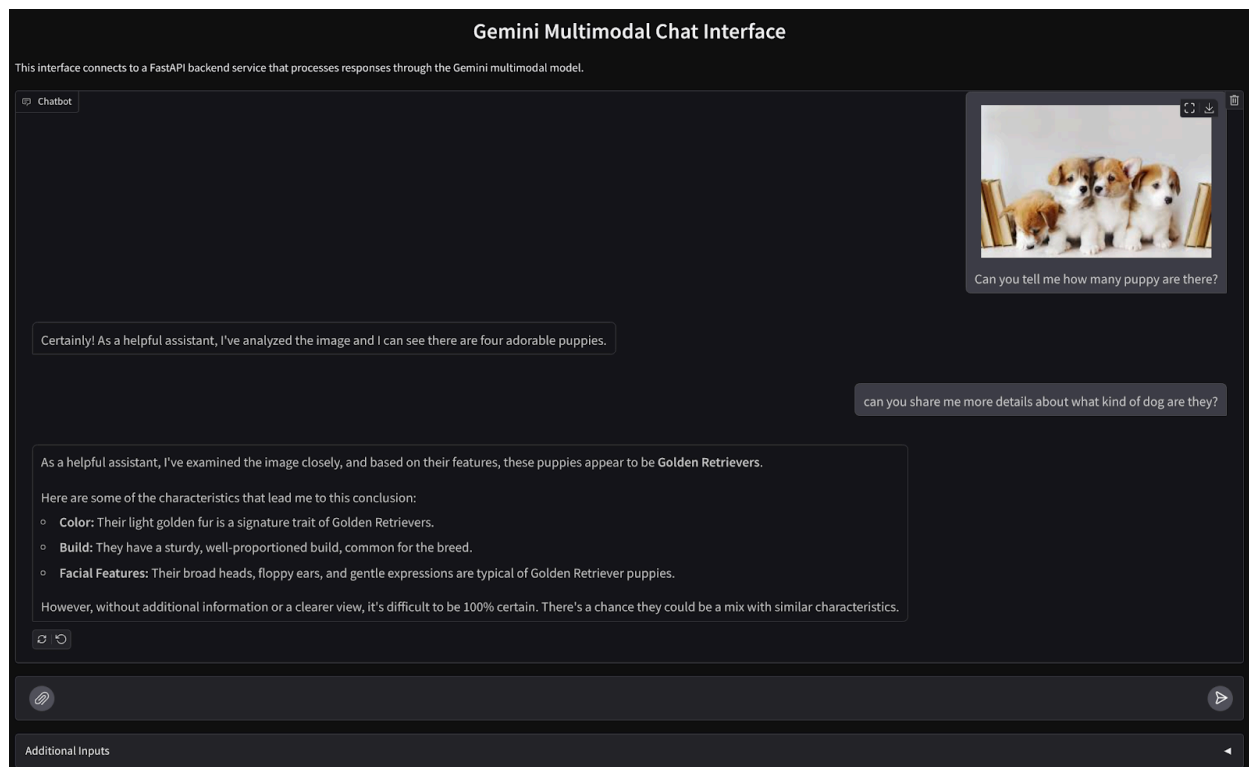
這些設定可讓我們彈性更新執行階段。在初始部署作業中，我們會依賴 **settings.yaml** 設定，以便取得第一個預設設定。之後，我們就可以透過控制台彈性更新環境變數，並重新部署，因為環境變數的優先順序高於預設 YAML 設定

現在我們可以繼續下一個步驟，建構服務

步驟 3 · 約 0 分鐘

3. 使用 Gradio 建構前端服務

我們將建構如下所示的即時通訊網頁介面



其中包含輸入欄位，供使用者傳送文字和上傳檔案。此外，使用者也可以在額外輸入欄位中覆寫系統指令，並傳送至 Gemini API

我們將使用 [Gradio](#) 建構前端服務。將 `main.py` 重新命名為 `frontend.py`，然後使用下列程式碼覆寫程式碼

```

import gradio as gr
import requests
import base64
from pathlib import Path
from typing import List, Dict, Any
from settings import get_settings, DEFAULT_SYSTEM_PROMPT

settings = get_settings()

IMAGE_SUFFIX_MIME_MAP = {
    ".png": "image/png",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".heic": "image/heic",
    ".heif": "image/heif",
    ".webp": "image/webp",
}

DOCUMENT_SUFFIX_MIME_MAP = {
    ".pdf": "application/pdf",
}

def get_mime_type(filepath: str) -> str:
    """Get the MIME type for a file based on its extension.

    Args:
        filepath: Path to the file.

    Returns:
        str: The MIME type of the file.

    Raises:
        ValueError: If the file type is not supported.
    """
    filepath = Path(filepath)
    suffix = filepath.suffix

    # modify ".jpg" suffix to ".jpeg" to unify the mime type
    suffix = suffix if suffix != ".jpg" else ".jpeg"

    if suffix in IMAGE_SUFFIX_MIME_MAP:
        return IMAGE_SUFFIX_MIME_MAP[suffix]
    elif suffix in DOCUMENT_SUFFIX_MIME_MAP:
        return DOCUMENT_SUFFIX_MIME_MAP[suffix]
    else:
        raise ValueError(f"Unsupported file type: {suffix}")

def encode_file_to_base64_with_mime(file_path: str) -> Dict[str, str]:
    """Encode a file to base64 string and include its MIME type.

    Args:

```

file_path: Path to the file to encode.

Returns:

Dict[str, str]: Dictionary with 'data' and 'mime_type' keys.

"""

mime_type = get_mime_type(file_path)

with open(file_path, "rb") as file:

base64_data = base64.b64encode(file.read()).decode("utf-8")

return {"data": base64_data, "mime_type": mime_type}

def get_response_from_llm_backend(

message: Dict[str, Any],

history: List[Dict[str, Any]],

system_prompt: str,

) -> str:

"""Send the message and history to the backend and get a response.

Args:

message: Dictionary containing the current message with 'text' and optional 'files'

keys.

history: List of previous message dictionaries in the conversation.

system_prompt: The system prompt to be sent to the backend.

Returns:

str: The text response from the backend service.

"""

Format message and history for the API,

NOTES: in this example history is maintained by frontend service,

hence we need to include it in each request.

And each file (in the history) need to be sent as base64 with its mime type

formatted_history = []

for msg in history:

if msg["role"] == "user" and not isinstance(msg["content"], str):

For file content in history, convert file paths to base64 with MIME type

file_contents = []

encode_file_to_base64_with_mime(file_path)

for file_path in msg["content"]

]

formatted_history.append({"role": msg["role"], "content": file_contents})

else:

formatted_history.append({"role": msg["role"], "content": msg["content"]})

Extract files and convert to base64 with MIME type

files_with_mime = []

if uploaded_files := message.get("files", []):

for file_path in uploaded_files:

files_with_mime.append(encode_file_to_base64_with_mime(file_path))

Prepare the request payload

```

message["text"] = message["text"] if message["text"] != "" else " "
payload = {
    "message": {"text": message["text"], "files": files_with_mime},
    "history": formatted_history,
    "system_prompt": system_prompt,
}

# Send request to backend
try:
    response = requests.post(settings.BACKEND_URL, json=payload)
    response.raise_for_status() # Raise exception for HTTP errors

    result = response.json()
    if error := result.get("error"):
        return f"Error: {error}"

    return result.get("response", "No response received from backend")
except requests.exceptions.RequestException as e:
    return f"Error connecting to backend service: {str(e)}"

if __name__ == "__main__":
    demo = gr.ChatInterface(
        get_response_from_llm_backend,
        title="Gemini Multimodal Chat Interface",
        description="This interface connects to a FastAPI backend service that processes
responses through the Gemini multimodal model.",
        type="messages",
        multimodal=True,
        textbox=gr.MultimodalTextbox(file_count="multiple"),
        additional_inputs=[
            gr.Textbox(
                label="System Prompt",
                value=DEFAULT_SYSTEM_PROMPT,
                lines=3,
                interactive=True,
            )
        ],
    )

    demo.launch(
        server_name="0.0.0.0",
        server_port=8080,
    )

```

完成後，我們可以使用下列指令執行前端服務。別忘了將 **main.py** 檔案重新命名為 **frontend.py**

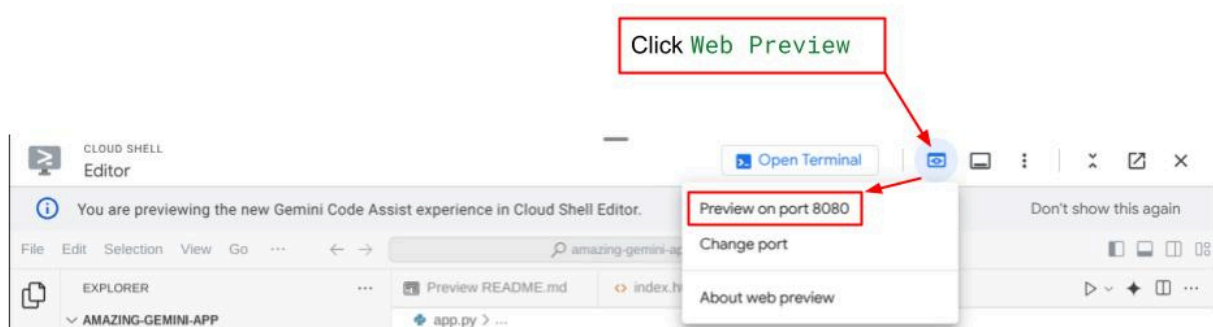
```
uv run frontend.py
```

雲端控制台會顯示類似以下的輸出內容

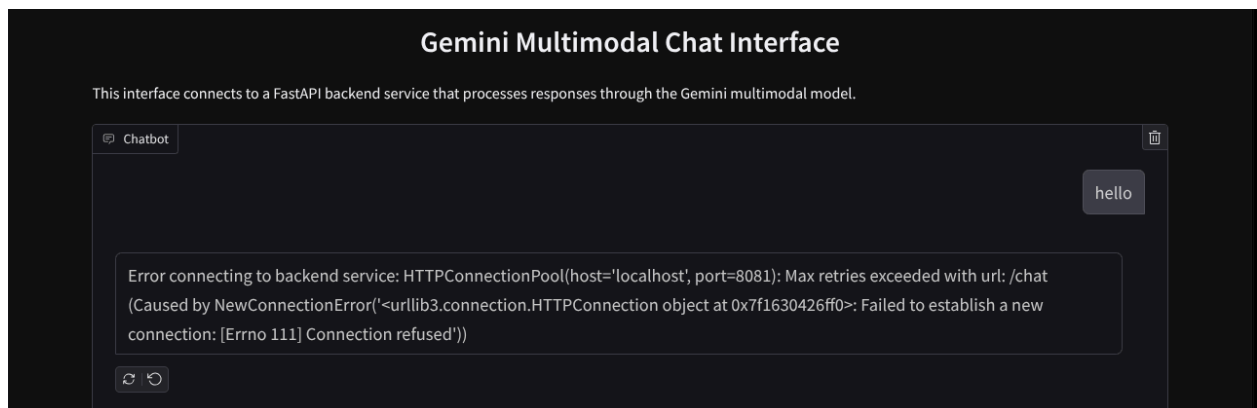
* Running on local URL: `http://0.0.0.0:8080`

To create a public link, set ``share=True`` in ``launch()``.

接著，只要 **按住 Ctrl 鍵並點選**本機網址連結，即可查看網頁介面。或者，您也可以點按 Cloud Editor 右上方的「Web Preview」（網頁預覽）按鈕，然後選取「Preview on port 8080」（透過以下通訊埠預覽：8080），存取前端應用程式。



您會看到網頁介面，但由於後端服務尚未設定，因此嘗試提交對話時會收到**預期錯誤**



現在請執行服務，但先別終止服務。在此期間，我們可以討論重要的程式碼元件

程式碼說明

將資料從網頁介面傳送至後端的程式碼位於這個部分

```

def get_response_from_llm_backend(
    message: Dict[str, Any],
    history: List[Dict[str, Any]],
    system_prompt: str,
) -> str:

    ...
    # Truncated

    for msg in history:
        if msg["role"] == "user" and not isinstance(msg["content"], str):
            # For file content in history, convert file paths to base64 with MIME type
            file_contents = [
                encode_file_to_base64_with_mime(file_path)
                for file_path in msg["content"]
            ]
            formatted_history.append({"role": msg["role"], "content": file_contents})
        else:
            formatted_history.append({"role": msg["role"], "content": msg["content"]})

    # Extract files and convert to base64 with MIME type
    files_with_mime = []
    if uploaded_files := message.get("files", []):
        for file_path in uploaded_files:
            files_with_mime.append(encode_file_to_base64_with_mime(file_path))

    # Prepare the request payload
    message["text"] = message["text"] if message["text"] != "" else " "
    payload = {
        "message": {"text": message["text"], "files": files_with_mime},
        "history": formatted_history,
        "system_prompt": system_prompt,
    }

    # Truncated
    ...

```

如要將多模態資料傳送至 Gemini，並讓服務之間可存取資料，其中一種機制是將資料轉換為程式碼中宣告的 [base64](#) 資料類型。我們也需要宣告資料的 [MIME](#) 類型。不過，Gemini API 無法支援所有現有的 MIME 類型，因此請務必參閱這份[文件](#)，瞭解 Gemini 支援哪些 MIME 類型。您可以在各項 Gemini API 功能 (例如[Vision](#)) 中找到相關資訊。

此外，在即時通訊介面中，將即時通訊記錄做為額外脈絡傳送給 Gemini，讓 Gemini 「記住」對話內容，也是非常重要的一環。因此在這個網頁介面中，我們也會傳送 Gradio 依網頁工作階段管理的即時通訊記錄，並與使用者輸入的訊息一併傳送。此外，我們也允許使用者修改系統指令並傳送

步驟 4 · 約 0 分鐘

4. 使用 FastAPI 建構後端服務

接著，我們需要建構後端，處理先前討論的酬載、最後一則使用者訊息、即時通訊記錄和系統指令。我們將使用 [FastAPI](#) 建立 HTTP 後端服務。

建立新檔案，依序點選「File」>「New Text File」，然後複製並貼上下列程式碼，並儲存為 **backend.py**

```

import base64
from fastapi import FastAPI, Body
from google.genai.types import Content, Part
from google.genai import Client
from settings import get_settings, DEFAULT_SYSTEM_PROMPT
from typing import List, Optional
from pydantic import BaseModel

app = FastAPI(title="Gemini Multimodal Service")

settings = get_settings()
GENAI_CLIENT = Client(
    location=settings.VERTEXAI_LOCATION,
    project=settings.VERTEXAI_PROJECT_ID,
    vertexai=True,
)
GEMINI_MODEL_NAME = "gemini-2.0-flash-001"

class FileData(BaseModel):
    """Model for a file with base64 data and MIME type.

    Attributes:
        data: Base64 encoded string of the file content.
        mime_type: The MIME type of the file.
    """

    data: str
    mime_type: str

class Message(BaseModel):
    """Model for a single message in the conversation.

    Attributes:
        role: The role of the message sender, either 'user' or 'assistant'.
        content: The text content of the message or a list of file data objects.
    """

    role: str
    content: str | List[FileData]

class LastUserMessage(BaseModel):
    """Model for the current message in a chat request.

    Attributes:
        text: The text content of the message.
        files: List of file data objects containing base64 data and MIME type.
    """

    text: str

```

```

files: List[FileData] = []

class ChatRequest(BaseModel):
    """Model for a chat request.

    Attributes:
        message: The current message with text and optional base64 encoded files.
        history: List of previous messages in the conversation.
        system_prompt: Optional system prompt to be used in the chat.
    """

    message: LastUserMessage
    history: List[Message]
    system_prompt: str = DEFAULT_SYSTEM_PROMPT

class ChatResponse(BaseModel):
    """Model for a chat response.

    Attributes:
        response: The text response from the model.
        error: Optional error message if something went wrong.
    """

    response: str
    error: Optional[str] = None

def handle_multimodal_data(file_data: FileData) -> Part:
    """Converts Multimodal data to a Google Gemini Part object.

    Args:
        file_data: FileData object with base64 data and MIME type.

    Returns:
        Part: A Google Gemini Part object containing the file data.
    """
    data = base64.b64decode(file_data.data) # decode base64 string to bytes
    return Part.from_bytes(data=data, mime_type=file_data.mime_type)

def format_message_history_to_gemini_standard(
    message_history: List[Message],
) -> List[Content]:
    """Converts message history format to Google Gemini Content format.

    Args:
        message_history: List of message objects from the chat history.
        Each message contains 'role' and 'content' attributes.

    Returns:

```

List[Content]: A list of Google Gemini Content objects representing the chat history.

Raises:

ValueError: If an unknown role is encountered in the message history.

"""

```
converted_messages: List[Content] = []
```

```
for message in message_history:
```

```
    if message.role == "assistant":
```

```
        converted_messages.append(
```

```
            Content(role="model", parts=[Part.from_text(text=message.content)])
```

```
        )
```

```
    elif message.role == "user":
```

```
        # Text-only messages
```

```
        if isinstance(message.content, str):
```

```
            converted_messages.append(
```

```
                Content(role="user", parts=[Part.from_text(text=message.content)])
```

```
            )
```

```
        # Messages with files
```

```
        elif isinstance(message.content, list):
```

```
            # Process each file in the list
```

```
            parts = []
```

```
            for file_data in message.content:
```

```
                for file_data in message.content:
```

```
                    parts.append(handle_multimodal_data(file_data))
```

```
            # Add the parts to a Content object
```

```
            if parts:
```

```
                converted_messages.append(Content(role="user", parts=parts))
```

```
        else:
```

```
            raise ValueError(f"Unexpected content format: {type(message.content)}")
```

```
    else:
```

```
        raise ValueError(f"Unknown role: {message.role}")
```

```
return converted_messages
```

```
@app.post("/chat", response_model=ChatResponse)
```

```
async def chat(
```

```
    request: ChatRequest = Body(...),
```

```
) -> ChatResponse:
```

```
    """Process a chat request and return a response from Gemini model.
```

Args:

request: The chat request containing message and history.

Returns:

ChatResponse: The model's response to the chat request.

"""

```
try:
```

```

# Convert message history to Gemini `history` format
print(f"Received request: {request}")
converted_messages = format_message_history_to_gemini_standard(request.history)

# Create chat model
chat_model = GENAI_CLIENT.chats.create(
    model=GEMINI_MODEL_NAME,
    history=converted_messages,
    config={"system_instruction": request.system_prompt},
)

# Prepare multimodal content
content_parts = []

# Handle any base64 encoded files in the current message
if request.message.files:
    for file_data in request.message.files:
        content_parts.append(handle_multimodal_data(file_data))

# Add text content
content_parts.append(Part.from_text(text=request.message.text))

# Send message to Gemini
response = chat_model.send_message(content_parts)
print(f"Generated response: {response}")

return ChatResponse(response=response.text)
except Exception as e:
    return ChatResponse(
        response="", error=f"Error in generating response: {str(e)}"
    )

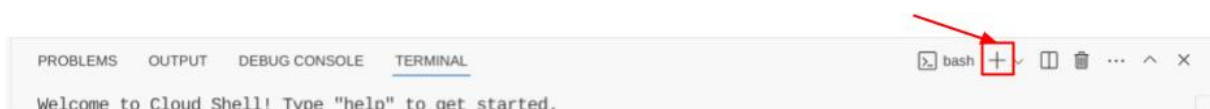
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8081)

```

別忘了將其儲存為 **backend.py**。接著，我們可以嘗試執行後端服務。請注意，我們在上一個步驟中執行了前端服務，現在需要開啟新的終端機，並嘗試執行這個後端服務

1. 建立新終端機。前往底部的終端機，然後找到「+」按鈕，即可建立新的終端機。或者，您也可以按下 **Ctrl + Shift + C** 開啟新的終端機



2. 接著，請確認您位於工作目錄 **gemini-multimodal-chat-assistant**，然後執行下列指令

```
uv run backend.py
```

3. 如果成功，輸出內容會如下所示

```
INFO:      Started server process [xxxxxx]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

程式碼說明

定義 HTTP 路由以接收即時通訊要求

在 FastAPI 中，我們使用 **app** 修飾符定義路徑。我們也使用 Pydantic 定義 API 規範。我們指定產生回覆的路徑為 **/chat** 路徑，並使用 **POST** 方法。下列程式碼中宣告的功能

```
class FileData(BaseModel):
    data: str
    mime_type: str

class Message(BaseModel):
    role: str
    content: str | List[FileData]

class LastUserMessage(BaseModel):
    text: str
    files: List[FileData] = []

class ChatRequest(BaseModel):
    message: LastUserMessage
    history: List[Message]
    system_prompt: str = DEFAULT_SYSTEM_PROMPT

class ChatResponse(BaseModel):
    response: str
    error: Optional[str] = None

    ...

@app.post("/chat", response_model=ChatResponse)
async def chat(
    request: ChatRequest = Body(...),
) -> ChatResponse:

    # Truncated
    ...
```

準備 Gemini SDK Chat 記錄格式

其中一項重要事項是瞭解如何重組即時通訊記錄，以便在稍後初始化 Gemini 用戶端時，將其插入為 **history** 引數值。您可以檢查下方的程式碼


```

def format_message_history_to_gemini_standard(
    message_history: List[Message],
) -> List[Content]:

    ...
    # Truncated

    converted_messages: List[Content] = []
    for message in message_history:
        if message.role == "assistant":
            converted_messages.append(
                Content(role="model", parts=[Part.from_text(text=message.content)])
            )
        elif message.role == "user":
            # Text-only messages
            if isinstance(message.content, str):
                converted_messages.append(
                    Content(role="user", parts=[Part.from_text(text=message.content)])
                )

            # Messages with files
            elif isinstance(message.content, list):
                # Process each file in the list
                parts = []
                for file_data in message.content:
                    parts.append(handle_multimodal_data(file_data))

                # Add the parts to a Content object
                if parts:
                    converted_messages.append(Content(role="user", parts=parts))

    #Truncated
    ...

    return converted_messages

```

如要將即時通訊記錄提供給 Gemini SDK，我們需要以 **List[Content]** 資料類型格式化資料。每個 **Content** 至少要有 **role** 和 **parts** 值。**role** 是指訊息來源，可以是 **user** 或 **model**。其中 **parts** 是指提示詞本身，可以是純文字，也可以是不同模態的組合。如要進一步瞭解如何建構 **Content** 引數，請參閱這份[說明文件](#)

處理非文字 (多模態) 資料

如先前在前端部分所述，傳送非文字或多模態資料的方法之一，就是將資料傳送為 **base64** 字串。我們也需要指定資料的 MIME 類型，才能正確解讀資料。舉例來說，如果傳送的圖片資料帶有 **.jpg** 後置字元，請提供 **image/jpeg** MIME 類型。

這部分程式碼會將 **base64** 資料轉換為 Gemini SDK 的 **Part.from_bytes** 格式

```
def handle_multimodal_data(file_data: FileData) -> Part:
    """Converts Multimodal data to a Google Gemini Part object.

    Args:
        file_data: FileData object with base64 data and MIME type.

    Returns:
        Part: A Google Gemini Part object containing the file data.
    """
    data = base64.b64decode(file_data.data) # decode base64 string to bytes
    return Part.from_bytes(data=data, mime_type=file_data.mime_type)
```

步驟 5 · 約 0 分鐘

5. 整合測試

現在，您應該會在不同的 Cloud 控制台分頁中執行多項服務：

- 前端服務在通訊埠 8080 執行

```
* Running on local URL:  http://0.0.0.0:8080

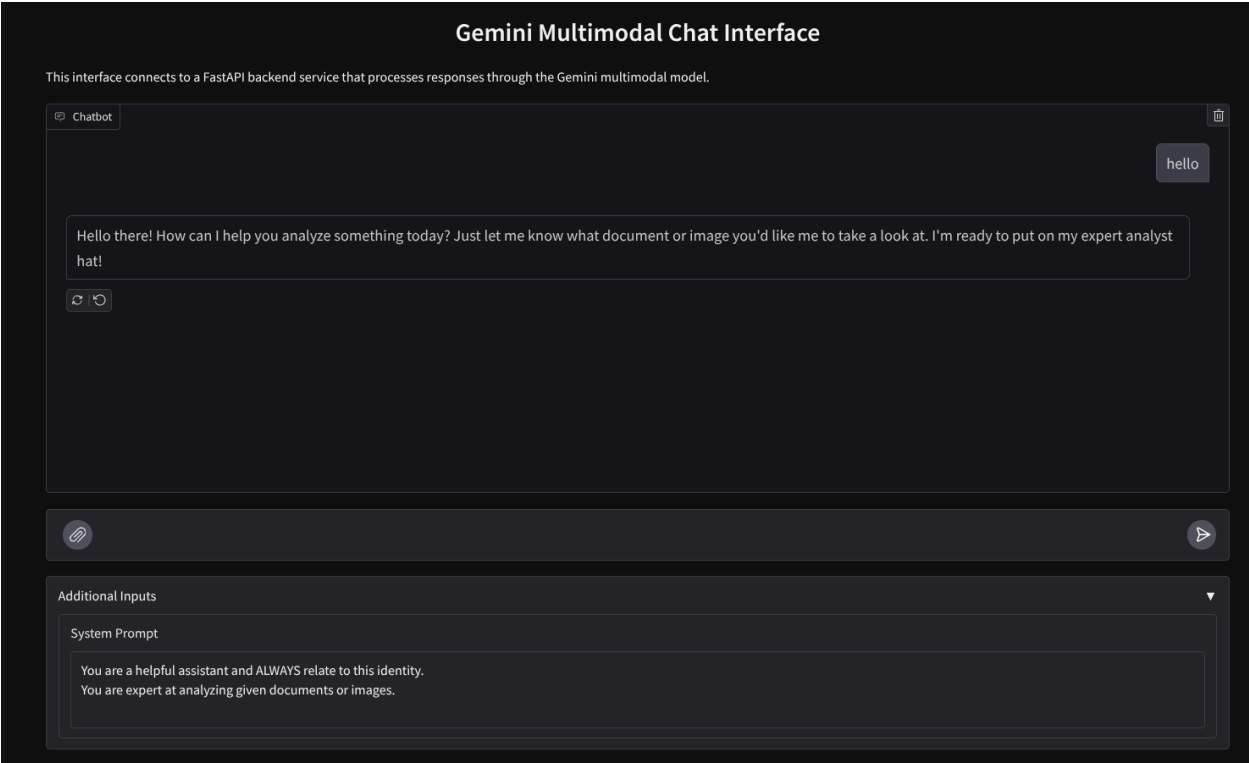
To create a public link, set `share=True` in `launch()`.
```

- 後端服務在通訊埠 8081 執行

```
INFO:      Started server process [xxxxxx]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

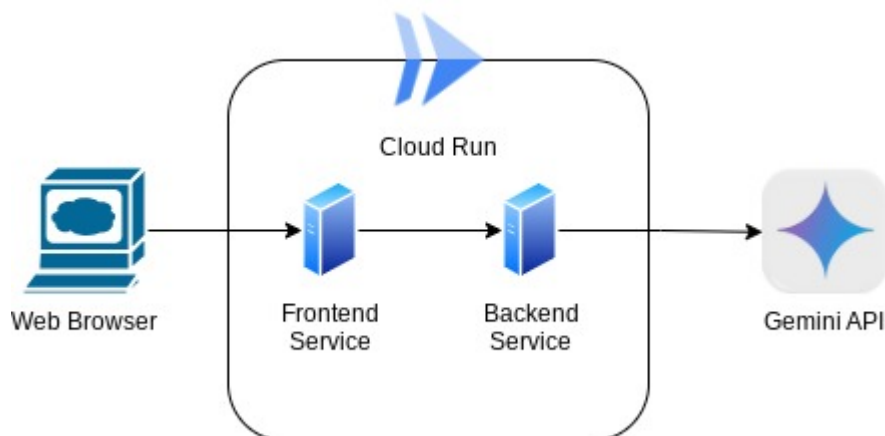
目前，您應該可以透過通訊埠 8080 的網頁應用程式，在即時通訊中輕鬆傳送文件給助理。上傳檔案並提問，即可開始實驗！請注意，**目前不支援某些檔案類型**，因此會引發錯誤。

您也可以在文字方塊下方的「其他輸入內容」欄位中編輯系統指示



6. 正在部署到 Cloud Run

現在，我們當然要向其他人展示這個出色的應用程式。為此，我們可以將這個應用程式封裝並部署至 Cloud Run，做為可供他人存取的公開服務。為此，我們來回顧一下架構



在本程式碼研究室中，我們將前端和後端服務都放在 1 個容器中。我們需要 [supervisord](#) 的協助，才能管理這兩項服務。

建立新檔案，依序點選「File」>「New Text File」，然後複製並貼上下列程式碼，並儲存為 **supervisord.conf**

```
[supervisord]
nodaemon=true
user=root
logfile=/dev/stdout
logfile_maxbytes=0
pidfile=/var/run/supervisord.pid

[program:backend]
command=uv run backend.py
directory=/app
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
stderr_logfile=/dev/stderr
stderr_logfile_maxbytes=0
startsecs=10
startretries=3

[program:frontend]
command=uv run frontend.py
directory=/app
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
stderr_logfile=/dev/stderr
stderr_logfile_maxbytes=0
startsecs=10
startretries=3
```

接著，我們需要 Dockerfile，依序點選「File」>「New Text File」，然後複製並貼上下列程式碼，並儲存為 **Dockerfile**

```

FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:0.6.6 /uv /uvx /bin/

RUN apt-get update && apt-get install -y \
    supervisor curl \
    && rm -rf /var/lib/apt/lists/*

ADD . /app
WORKDIR /app

RUN uv sync --frozen

EXPOSE 8080

# Copy supervisord configuration
COPY supervisord.conf /etc/supervisor/conf.d/supervisord.conf

ENV PYTHONUNBUFFERED=1

ENTRYPOINT ["/usr/bin/supervisord", "-c", "/etc/supervisor/conf.d/supervisord.conf"]

```

此時，我們已備妥將應用程式部署至 Cloud Run 的所有必要檔案，現在就來部署吧。前往 Cloud Shell 終端機，確認目前專案已設為有效專案。如果不是，請使用 gcloud 設定指令設定專案 ID：

```
gcloud config set project [PROJECT_ID]
```

接著，執行下列指令將其部署至 Cloud Run。

```

gcloud run deploy --source . \
    --env-vars-file settings.yaml \
    --port 8080 \
    --region us-central1

```

系統會提示您輸入服務名稱，假設為「**gemini-multimodal-chat-assistant**」。由於應用程式工作目錄中有 Dockerfile，系統會建構 Docker 容器並推送至 Artifact Registry。系統也會提示您將在該區域建立 Artifact Registry 存放區，請回答「Y」。系統詢問是否要允許未經驗證的叫用要求時，請輸入「y」。請注意，我們在這裡允許未經驗證的存取要求，因為這是示範應用程式。建議您為企業和正式版應用程式使用適當的驗證方式。

部署完成後，您會取得類似下方的連結：

[https://gemini-multimodal-chat-assistant-***.us-central1.run.app](https://gemini-multimodal-chat-assistant-*****.us-central1.run.app)**

請繼續在無痕視窗或行動裝置上使用應用程式。這項功能應該已經上線。

步驟 7 · 約 0 分鐘

7. 競賽項目

現在輪到你大顯身手，磨練探索技能。你有能力變更程式碼，讓助理支援讀取音訊檔案或影片檔案嗎？

步驟 8 · 約 0 分鐘

8. 清理

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
 4. 或者，您也可以前往控制台的「Cloud Run」，選取剛部署的服務並刪除。
-